SMARTMOVE TRANSPORT SOLUTIONS

HIGHER NATIONAL DIPLOMA IN SOFTWARE ENGINEERING
26.1F

DATA MANAGEMENT 2
COURSEWORK REPORT



NAME	INDEX
R.G.C Madushan	COHNDSE261F-035
M.T.M Perera	COHNDSE261F-015
Oshan Tanusha	COHNDSE261F-039
ACMJ. Aarif	COHNDSE261F-016

Declaration

We hereby declare that this coursework report, titled SmartMove: Transport Management System, is our own original work,done as part of the Data Management Systems (DM2) module at the National Institute of Business Management (NIBM). This project has been carried out in accordance with the guidelines provided by the university, and all work presented in this report, including the database design, SQL and PL/SQL implementation, MongoDB document modeling, web application development, and evaluation, has been completed by our group members. We confirm that this report has not been submitted for any other academic requirements or purpose. All sources and references used throughout this report have been resoectfully acknowledged.

Contents

Data Management Systems....0
HIGHER NATIONAL DIPLOMA IN SOFTWARE ENGINEERING....0
ACKNOWLEDGEMENT....3
1. INTRODUCTION....4
1.1 Project Background....4
1.2 Problem Statement....4
1.3 Project Aim....5
1.4 Project Objectives....5
1.5 Project Scope....5
2. REQUIREMENTS....6
2.1 Functional Requirements....6
2.2 Non-Functional Requirements....7
2.3 Hardware and Software Requirements....8
3. SYSTEM DESIGN....9
3.1 System Architecture....9
3.2 Main System Modules....10
3.3 Data Flow....11
4. ENTITY RELATIONSHIP DIAGRAM (ERD)....12
4.1 Relationship Descriptions....12
4.2 Integrity Constraints....13
5. ORACLE DATABASE....14
5.1 Database Tables....14
5.2 Data Types....15
5.3 Sample Data....15
5.4 What to Include in the Report (Screenshots Checklist)....15
6. SQL (STRUCTURED QUERY LANGUAGE)....16
6.1 Data Definition Language (DDL)....16
6.2 Data Manipulation Language (DML)....16

6.3 Data Retrieval and Reporting....16
6.4 SQL Features Explained....17
7. PL/SQL (PROCEDURAL LANGUAGE / SQL)....18
7.1 Stored Procedures....18
7.2 Stored Functions....18
7.3 Cursors....19
7.4 Exception Handling....19
7.5 Testing PL/SQL....19
8. MONGODB....20
8.1 Database and Collections....20
8.2 Indexing....20
8.3 Queries and Aggregation....21
8.4 Update and Delete Operations....21
8.5 MongoDB Evidence....21
9. WEB APPLICATION....22
9.1 Frontend Interface....22
9.2 Backend API....22
9.3 Application Features....22
9.4 Screenshots to Include (Application Walkthrough)....22
10. DATABASE INTEGRATION....23
10.1 Oracle Integration....23
10.2 MongoDB Integration....23
10.3 Request and Response Lifecycle....23
10.4 Integration Testing....23
11. TESTING....24
11.1 Test Plan....24
11.2 Testing Methods....25
11.3 Recording the Results....26
12. RESULTS....27

12.1 Database Results..... 27
12.2 PL/SQL Results..... 27
12.3 MongoDB Results..... 27
12.4 Application & Integration Results..... 27
13. DISCUSSION AND CONCLUSION..... 29
13.1 Real Project Experiences & Challenges..... 29
13.2 Limitations & Lessons Learned..... 29
13.3 Future Work..... 30
14. REFERENCES..... 31
APPENDICES..... 32
Appendix A: Oracle Table Creation Scripts..... 32
Appendix B: SQL Queries and Execution Outputs..... 32
Appendix C: PL/SQL Procedures, Functions, Cursors, and Exception Tests..... 32
Appendix D: MongoDB Setup, Insertion, Query, and Aggregation Scripts..... 32
Appendix E: Application Screenshots..... 32
Appendix F: Integration and Verification Evidence..... 32
Appendix G: GitHub Repository Link..... 32

ACKNOWLEDGEMENT

We would like to take this opportunity to thank our module lecturer at the National Institute of Business Management (NIBM) for giving us the guidance, technical support, and constructive feedback throughout this DM2 coursework project. Their lectures on relational database design, PL/SQL programming, and NoSQL technologies gave us a clear direction on how to structure our hybrid architecture properly.

We would also like to thank our batchmates in 26.1FT for discussing database connection issues and debugging ideas with us during practical sessions. This project was a joint group effort, with each member contributing to database scripts, backend PHP code, frontend design, and testing.

1. INTRODUCTION

1.1 Project Background

In Sri Lanka, most private bus operators and inter city van providers still manage their daily operations manually. They use paper receipt books, phone calls, and basic Excel sheets. When a business only has one or two vans running between Colombo and Galle, that might be fine. But as soon as the company adds more vehicles, routes, and drivers, things get messy very quickly. Common issues happen all the time:

- Two different ticket counters book the same window seat for two passengers on the same bus trip.
- A bus that is stuck in the workshop for brake repairs gets accidentally scheduled for a long-distance night trip.
- Fleet managers have to call drivers repeatedly to check who is on duty and who is on leave.
- Ticket clerks take cash fares, but at the end of the week, the total cash in hand doesn't match the passenger manifest records.

We created SmartMove Transport Solutions to solve these problems. It is a centralized system where fleet managers can manage vehicles, drivers, route schedules, passenger tickets, payments, customer reviews, and maintenance logs in one place.

1.2 Problem Statement

Transport operations deal with two completely different kinds of data:

1. **Core transactional data:** Bookings, ticket seats, fares, payment records, vehicle assignments, and maintenance costs. This information must be accurate. If two customers try to book seat number 12 on the 8:00 AM trip from Colombo to Galle, the database must stop the second booking on the spot. Foreign keys and check constraints are extremely important here here.

2. **Flexible, document-based data:** Passenger reviews and ratings, emergency route delay announcements, scanned vehicle insurance documents, and photos of road conditions or vehicles. These don't have a rigid structure. For example, some customer reviews have photos and long text, while others just have a rating. Some vehicles have five different inspection files, while others only have two.

If you try to stuff everything into Oracle relational tables, you end up with messy tables filled with empty columns. On the flip side, if you only use MongoDB, you lose out on Oracle's rock solid foreign keys, ACID transactions, and PL/SQL procedural checks for seat reservations. Our team adopted a hybrid database model: Oracle Database handles the structured relational operations, while MongoDB takes care of the flexible documents.

1.3 Project Aim

Our aim is to build a practical, integrated transport management system that combines a normalized Oracle relational database for transactional data, PL/SQL code for business rules, MongoDB collections for reviews and documents, and an easy to use web interface built with PHP and JavaScript.

1.4 Project Objectives

Design a clean, normalized relational schema modeling 9 transport operational entities.
Implement primary keys, foreign keys, unique rules, and domain check constraints inside Oracle Database.

Write SQL queries for daily retrieval, fleet grouping, and route booking reports.
Build stored procedures, functions, explicit cursors, and custom exception handlers using PL/SQL.

Set up 4 MongoDB collections with indexes for passenger reviews, announcements, vehicle paperwork, and trip media.

Write MongoDB aggregation pipelines to rank vehicles by rating, calculate route averages, and count keywords.

Build an administrative web interface using HTML, CSS, and JavaScript

1.5 Project Scope

The project covers managing vehicles, drivers, routes, scheduled trips, customer accounts, seat bookings, payment records, trip reviews, and maintenance tracking. It also covers document storage, keyword search in reviews, and database integration checks.

2. REQUIREMENTS

2.1 Functional Requirements

ID	Requirement	What It Does in Our System
FR01	Passenger Management	Lets administrators register new passengers and update their phone numbers and emails.
FR02	Vehicle Management	Allows adding buses and vans, setting seating capacities, and updating status (Active, Maintenance, Retired).
FR03	Driver Management	Stores driver names, commercial license numbers, phone numbers, and duty availability.
FR04	Route Management	Defines routes with starting points, destinations, and official distances in kilometers.
FR05	Trip Scheduling	Schedules journeys by picking a route, an active vehicle, an available driver, and departure/arrival times.
FR06	Booking and Ticketing	Reserves specific seats for passengers, checks availability, and handles ticket cancellations.
FR07	Payment Processing	Logs payment amounts, dates, payment types (Cash, Card, Online Transfer), and settlement status.
FR08	Passenger Feedback	Captures star ratings (1 to 5) and customer comments for completed journeys.
FR09	Fleet Maintenance	Records maintenance dates, repair descriptions, garage bills, and repair completion status.
FR10	Document & Media Storage	Stores vehicle insurance documents, fitness certificates, travel alerts, and trip photos in MongoDB.
FR11	Feedback Search	Allows searching through passenger review text using keywords like "clean", "delayed", or "polite".
FR12	Operational Reporting	Produces management summaries showing vehicle status counts, route booking totals, and revenue.
FR13	Web Administration Interface	Provides a clean web portal where staff can do all daily operations without writing SQL manually.

2.2 Non-Functional Requirements

- **Data Integrity:** Foreign keys and check constraints ensure bad data (like negative seat numbers or duplicate registration plates) is blocked at the database
- **Usability:** Simple navigation tabs so clerks and managers can find vehicles, bookings, and reports without needing training.
- **Reliability:** The system catches database connection drops gracefully and gives meaningful messages instead of blank error screens.
- **Input Validation:** Form inputs are validated on the client side with JavaScript
- **Maintainable Code:** Frontend files, backend endpoints, and database scripts are kept in separate folders so other developers can follow the structure easily.
- **Security:** Database passwords are stored in a private .env file instead of hardcoded. Prepared statements prevent SQL injection attacks.
- **Speed & Performance:** Queries run quickly because primary keys, foreign keys, and MongoDB search fields use dedicated indexes.

2.3 Hardware and Software Requirements

Development Hardware: Laptop with Intel Core i5 or AMD Ryzen, 8 GB RAM (16 GB recommended to run Oracle XE and MongoDB together), and 25 GB free disk space.

Database Software:

Oracle Database 19c / 21c Express Edition (XE)
Oracle SQL Developer & Data Modeler
MongoDB Community Server (v6.0+)
MongoDB Compass and mongosh terminal shell

Web Development Tools:

Visual Studio Code
PHP 8.1+ with OCI8 and MongoDB extension
Apache Web Server (via XAMPP)
Google Chrome / Firefox for testing
Git & GitHub for version control

3. SYSTEM DESIGN

3.1 System Architecture

We used a standard layered architecture to keep our project clean and organized:

Architectural Tier	Tools & Technologies	Role in Our Application
Presentation Tier	HTML5, CSS3, JavaScript	Web pages that clerks and managers use. Handles button clicks, form submissions, and table updates.
Application Tier	PHP 8.x JSON API	Receives requests from JavaScript, checks inputs, talks to Oracle or MongoDB, and returns JSON.
Relational Database Tier	Oracle Database (SQL, PL/SQL)	Stores all relational records, enforces foreign keys, and runs stored procedures for seat booking.
Document Database Tier	MongoDB (BSON Collections)	Stores flexible customer reviews, service notices, vehicle PDF/image metadata, and media URLs.

3.2 Main System Modules

Module Name	Purpose in Daily Operations
Vehicle Management	Keep track of vehicle registration numbers, model, seating capacity, and whether it's available or broken down.
Driver Management	Store driver names, contact numbers, license numbers, and check if they are on duty or off duty.
Route Management	Store origin, destination, and distance in km for instance Colombo to Kandy: 115 km.
Trip Management	Create daily bus timetables by assigning a route, vehicle, driver, and departure/arrival times.
Booking Management	Let passengers book seats, calculate ticket fares, stop double-bookings, and handle ticket cancellations.
Payment Management	Track whether tickets were paid by cash, credit card, or bank transfer, and record payment dates.
Feedback Management	Collect 1-to-5 star ratings and written reviews from passengers after their trip finishes.
Maintenance Management	Track when a bus visited the workshop, what repairs were done for example brake pads, oil change, and the repair cost.
Document & Media Management	Store metadata and links for vehicle insurance certificates, revenue licenses, road alerts, and trip photos.
Operational Reporting	Provide filterable summary reports on fleet availability, popular routes, passenger history, and earnings.

3.3 Data Flow

Here is what happens when someone performs an action in SmartMove:

1. The user fills out a form in the web browser and clicks submit.
2. JavaScript validates that the inputs aren't empty and sends a POST request with JSON data to backend/backend.php.
3. The PHP script receives the request, verifies user permissions, and decides which database to query.
4. For booking a seat, PHP connects to Oracle via OCI8 and executes the CreateBooking stored procedure.
5. Oracle verifies the seat is empty, adds the row to BOOKING, and returns success (or raises an error if already booked).
6. PHP takes the result from Oracle, puts it in a neat JSON response ({"success": true, "message": "Booking confirmed"}), and sends it back.
7. JavaScript updates the screen, turns the seat red (booked), and shows a green success banner.

4. ENTITY RELATIONSHIP DIAGRAM (ERD)

Our Oracle relational database contains nine core tables defined in oracle/create_tables.sql:

- VEHICLE- (VehicleID [PK], RegistrationNo, Capacity, VehicleStatus)
- DRIVER- (DriverID [PK], LicenseNo, FirstName, LastName, Phone, DriverStatus)
- ROUTE -(RouteID [PK], StartLocation, EndLocation, DistanceKM, RouteStatus)
- PASSENGER -(PassengerID [PK], FirstName, LastName, Phone, Email)
- TRIP -(TripID [PK], VehicleID [FK], DriverID [FK], RouteID [FK], TripDate, DepartureTime, ArrivalTime, TripStatus)
- BOOKING -(BookingID [PK], PassengerID [FK], TripID [FK], SeatNumber, Fare, BookingStatus, BookingDate)
- PAYMENT- (PaymentID [PK], BookingID [FK], Amount, PaymentMethod, PaymentDate, PaymentStatus)
- FEEDBACK- (FeedbackID [PK], PassengerID [FK], TripID [FK], Rating, CommentText, FeedbackDate)
- MAINTENANCE- (MaintenanceID [PK], VehicleID [FK], MaintenanceDate, Description, Cost, Status)

4.1 Relationship Descriptions

Relationship	Cardinality	Explanation
Vehicle to Trip	1 to Many (1:M)	One vehicle is assigned to many different scheduled trips over time.
Driver to Trip	1 to Many (1:M)	One driver drives multiple scheduled trips across different days.
Route to Trip	1 to Many (1:M)	One route (e.g., Colombo to Galle) is used by many scheduled trips each day.
Passenger to Booking	1 to Many (1:M)	One passenger can book seats across many different trips over time.
Trip to Booking	1 to Many (1:M)	One trip has multiple seat bookings from passengers up to vehicle capacity.
Booking to Payment	1 to 1 (1:1)	Each booking record has at most one payment record linked directly to it.
Passenger to Feedback	1 to Many (1:M)	One passenger can submit reviews for different trips they traveled on.
Trip to Feedback	1 to Many (1:M)	One trip can receive reviews from multiple passengers who were on board.
Vehicle to Maintenance	1 to Many (1:M)	One vehicle can have multiple service and repair logs in the garage over its lifetime.

4.2 Integrity Constraints

We spent considerable time setting up constraints so that the database protects itself from bad data:

- **Primary Key Constraints:** Every table uses Oracle identity columns (GENERATED ALWAYS AS IDENTITY) to create unique numbers automatically.
- **Foreign Key Constraints:** Defined between TRIP and its parent tables (VEHICLE, DRIVER, ROUTE), and between BOOKING and (PASSENGER, TRIP). If you try to delete a vehicle that has scheduled trips, Oracle stops you.
- **Unique Constraints:**
    - Vehicle registration plate (RegistrationNo) is unique so no two buses have the same plate.
    - Driver license number (LicenseNo) is unique.
    - Passenger email (Email) is unique to prevent duplicate accounts.
    - Composite unique key (TripID, SeatNumber) in BOOKING guarantees that two passengers can never book the exact same seat on the exact same trip.
- **Check Constraints:**
    - Capacity > 0 and SeatNumber > 0 stop negative numbers.
    - Rating BETWEEN 1 AND 5 ensures star ratings stay between 1 and 5.
    - DistanceKM > 0 and Cost >= 0 ensure valid positive distances and repair bills.
    - Status columns are restricted to fixed words, like VehicleStatus IN ('Active', 'Maintenance', 'Retired').
    - ArrivalTime >= DepartureTime ensures arrival time is never earlier than departure.

5. ORACLE DATABASE

5.1 Database Tables

Table Name	What We Use It For
VEHICLE	Holds bus and van details: registration plate, vehicle type, passenger capacity, and status.
DRIVER	Holds driver records: full name, driving license number, mobile phone, and active status.
ROUTE	Holds route details: starting town, terminus town, road distance in kilometers, and status.

Table Name	What We Use It For
PASSENGER	Holds customer records: first name, last name, mobile phone number, and unique email.
TRIP	Holds daily schedules: ties together a vehicle, a driver, a route, trip date, and departure/arrival times.
BOOKING	Holds ticket reservations: links passenger to trip, records seat number, fare price, and booking status.
PAYMENT	Holds financial transactions: payment amount, payment method, and settlement status.
FEEDBACK	Holds trip ratings and customer remarks tied to a specific passenger and trip.
MAINTENANCE	Holds vehicle repair logs: date in workshop, mechanic work description, total repair cost, and status.

5.2 Data Types

We selected standard Oracle data types based on field requirements:

- NUMBER: Used for surrogate IDs (identity columns), passenger counts, ratings, and money amounts (NUMBER(10,2)).
- VARCHAR2: Used for text strings like passenger names, registration plates, emails, and comments.
- DATE: Used for calendar dates like trip dates, booking dates, and garage servicing dates.
- TIMESTAMP: Used for exact departure and arrival times where hours and minutes are critical.

- Identity Columns: Used on all primary keys to auto-generate sequential IDs without needing manual sequences.

5.3 Sample Data

We created a sample data script (oracle/sample_data.sql) containing realistic test records:

3 sample vehicles: VHC1001 (45-seater highway bus), VHC1002 (14-seater Kandy commuter van), and VHC1003 (28-seater coaster bus).

3 common Sri Lankan routes: Colombo to Galle (120 km via Southern Expressway), Colombo to Kandy (115 km), and Galle to Matara (45 km).

5 registered drivers, 10 passengers, 6 scheduled trips, and corresponding booking records.

6. SQL (STRUCTURED QUERY LANGUAGE)

6.1 Data Definition Language (DDL)

We wrote clean DDL statements in oracle/create_tables.sql with inline primary keys, foreign keys, and check constraints.

A quick test query to check vehicle records:

```
SELECT *
FROM VEHICLE;
```

6.2 Data Manipulation Language (DML)

We populated tables using INSERT INTO statements in oracle/sample_data.sql. In the web application, user actions trigger parameterized INSERT, UPDATE, and DELETE statements.

6.3 Data Retrieval and Reporting

We created two key management report queries that our web application runs to show operational summaries:

Query 1: Fleet Breakdown by Vehicle Status

```
SELECT VehicleStatus, COUNT(*) AS TotalVehicles
FROM VEHICLE
GROUP BY VehicleStatus
ORDER BY TotalVehicles DESC;
```

Why this query is useful: Fleet managers can see at a glance how many buses are ready to drive ('Active'), how many are broken down ('Maintenance'), and how many are taken out of service ('Retired').

Query 2: Total Bookings and Passenger Demand per Route

```
SELECT r.StartLocation,
    r.EndLocation,
    COUNT(b.BookingID) AS TotalBookings
FROM ROUTE r
JOIN TRIP t ON t.RouteID = r.RouteID
LEFT JOIN BOOKING b ON b.TripID = t.TripID
GROUP BY r.StartLocation, r.EndLocation
ORDER BY TotalBookings DESC;
```

Why this query is useful: It joins ROUTE, TRIP, and BOOKING using a LEFT JOIN so even routes with zero bookings still show up. This tells operators which corridors are making the most money.

6.4 SQL Features Explained

SELECT: Pulls out the exact columns and calculated totals we want.

INSERT: Adds new passengers, vehicles, and bookings into tables.

UPDATE: Changes existing records, such as setting a vehicle status to 'Maintenance'.

DELETE: Deletes records, while foreign keys prevent deleting parent records that have children.

JOIN / LEFT JOIN: Matches rows across tables based on foreign key IDs.

GROUP BY: Groups rows together so aggregate functions like COUNT() can summarize them.

ORDER BY: Sorts the final output neatly so top routes or vehicle counts appear first.

Constraints: Guard data integrity directly inside the database engine.

Note: Place screenshots of your SQL Developer query results and data grids here.

7. PL/SQL (PROCEDURAL LANGUAGE / SQL)

PL/SQL allowed us to write server side business logic directly inside Oracle, so data validation rules are always enforced even if someone bypasses the web interface.

7.1 Stored Procedures

Procedure Name	Where It Is Saved	What It Does
CreateBooking	plsql/procedures.sql	Checks if the passenger and trip exist, checks that the seat number isn't higher than vehicle capacity, verifies the seat isn't already taken, inserts the booking row, and raises an error if any check fails.
CancelBooking	plsql/procedures.sql	Looks up a booking ID, checks that it isn't already cancelled, updates BookingStatus to 'Cancelled', and throws an error if the booking ID doesn't exist.
ScheduleTrip	plsql/procedures.sql	Checks that the vehicle, driver, and route exist, verifies that the vehicle is marked 'Active' rather than 'Maintenance', and inserts the new trip timetable.
RegisterPassenger	plsql/procedures.sql	Inserts a new passenger profile and catches duplicate email errors cleanly.

7.2 Stored Functions

- **CalculateRevenue(p_StartDate IN DATE, p_EndDate IN DATE) RETURN NUMBER:** Adds up all completed payment amounts between two dates to calculate total revenue.

- **CountPassengerTrips(p_PassengerID IN NUMBER) RETURN NUMBER:** Counts how many confirmed trips a specific customer has taken.

- **GetAvailableSeats(p_TripID IN NUMBER) RETURN NUMBER:** Looks up the vehicle's capacity for a trip, subtracts the number of confirmed bookings, and returns how many seats are left.

Example of calling our seat calculation function:

```
SELECT GetAvailableSeats(1) AS AvailableSeats
FROM DUAL;
```

7.3 Cursors

- **PassengerTravelHistory(p_PassengerID IN NUMBER):** Uses an explicit cursor to loop through all bookings for a given passenger, printing their travel history, routes, and fares to the screen via DBMS_OUTPUT.
- **VehiclesDueForMaintenance:** Loops through all vehicles to find which ones have upcoming service dates or have exceeded mileage limits.
- **MostUsedRoutes:** Loops through route statistics to rank routes based on passenger ticket counts.

7.4 Exception Handling

We used structured exception handling across our PL/SQL scripts:

- **NO_DATA_FOUND:** Caught when searching for a trip, passenger, or booking ID that doesn't exist.
- **DUP_VAL_ON_INDEX:** Caught when an insert violates a unique key, like duplicate emails or duplicate registration numbers.
- **TOO_MANY_ROWS:** Handled when a single-row SELECT unexpectedly matches multiple records.
- **RAISE_APPLICATION_ERROR:** Used to return clear custom error messages (e.g., -20001: That seat is already booked for this trip).
- **WHEN OTHERS:** A safety net that logs unexpected errors and executes a ROLLBACK so transactions don't get left half-done.

7.5 Testing PL/SQL

We tested our procedures and functions using test scripts (procedure_tests.sql, function_tests.sql, cursors_test.sql, and exception_tests.sql). During initial testing, we spotted and fixed a typo in a DBMS_OUTPUT call in plsql/exceptions.sql before running the final tests.

8. MONGODB

We used MongoDB alongside Oracle to store semi-structured and document-based data that benefits from a flexible schema.

8.1 Database and Collections

Our MongoDB database is named smartmove_transport and has four collections:

Collection Name	What We Store in This Collection
vehicleDocuments	Vehicle compliance metadata: registration numbers, document types (Insurance, Fitness, Road Tax), issuing authorities, expiry dates, and document image paths.
reviewsFeedback	Customer feedback: star ratings (1 to 5), written comments, automated keyword tags, passenger IDs, route IDs, and timestamps.
announcements	Public service notices: route delay alerts, weather warnings, maintenance notices, priority levels, and publish dates.
tripMedia	Journey multimedia: photo and video URLs, MIME types, file sizes, captions, and upload timestamps linked to specific trips.

8.2 Indexing

To keep searches fast as data grows, we created five indexes:

- Unique index on { vehicleId: 1 } in vehicleDocuments so each vehicle only has one compliance document profile.
- Compound index on { routeId: 1, createdAt: -1 } in reviewsFeedback to quickly pull reviews for a specific route sorted by newest first.
- Text index on { comment: "text", keywords: "text" } in reviewsFeedback to enable full-text keyword searches across passenger comments.
- Compound index on { status: 1, publishDate: -1 } in announcements to quickly grab all currently active travel alerts.
- Index on { tripId: 1, uploadDate: -1 } in tripMedia to quickly load all photos taken on a specific journey.

8.3 Queries and Aggregation

We wrote query and aggregation scripts in queries.js and aggregation.js for our coursework requirements:

1. Retrieve reviews for a specific route.
2. Rank vehicles based on average customer ratings.

3. Rank drivers based on average customer ratings.
4. Search passenger comments using text keywords (e.g., "clean", "delayed", "comfortable").
5. Retrieve compliance documents and trip media for a specific vehicle.
6. Calculate average customer ratings grouped by route.
7. Count keyword occurrences across feedback records to identify recurring service issues.
8. Count media uploads per trip.

Our aggregation pipelines use $match, $group, $avg, $sum, $unwind, $addToSet, and $sort to compute clean summaries.

8.4 Update and Delete Operations

In updates.js, we demonstrated:

- Updating vehicle document expiry dates and appending renewal records.
- Adding new image URLs to vehicle documents using the $push operator.
- Deactivating older announcements by setting their status to 'Inactive'.
- Deleting specific media records from the tripMedia collection.

9. WEB APPLICATION

9.1 Frontend Interface

HTML Structure: Clean HTML5 (frontend/index.html) organized into separate view sections for fleet management, drivers, routes, trips, bookings, and reports.

CSS Styling: Written in frontend/css/styles.css, providing a modern responsive design with clean data tables, modal popups, and status pills.

JavaScript: Written in vanilla JavaScript (frontend/js/) to handle form validation, button clicks, dynamic table rendering, and asynchronous API calls using fetch().

9.2 Application Features

- Full CRUD management pages for Vehicles, Drivers, Routes, Trips, Passengers, and Bookings.
- Five dedicated searchable report screens: Routes, Payments, Passengers, Maintenance, and Drivers.
- A built-in database ping feature that tests Oracle and MongoDB connectivity with one click.

10. DATABASE INTEGRATION

10.1 Oracle Integration

Oracle handles all structured operational records. In our PHP backend, we established connections using oci_connect(), prepared statements using oci_parse(), bound parameters using oci_bind_by_name() to prevent SQL injection, and executed transactions with proper commit and rollback handling.

10.2 MongoDB Integration

MongoDB stores document-based information. We initialized the MongoDB client in PHP using new MongoDB\Client() and interacted with collections using standard methods like insertOne(), find(), updateOne(), and aggregate().

10.3 Request and Response Lifecycle

1. The user fills out a form or clicks an action button in the web interface.
2. Client-side JavaScript intercepts the event, validates required fields, and sends an asynchronous request with a JSON payload to backend/backend.php.
3. The PHP backend receives the payload, verifies session authentication, and validates parameters.
4. The backend routes the operation to the appropriate database:
    - Calling Oracle SQL queries or PL/SQL procedures via OCI8.
    - Calling MongoDB collection operations via the MongoDB library.
5. The database executes the command and returns result rows or status flags.
6. The PHP backend formats the output into a standardized JSON response ({ "success": true, "data": [...] }).
7. JavaScript receives the JSON response and updates the UI tables or displays an alert notification.

10.4 Integration Testing

To protect credentials, connection settings are configured in a private .env file rather than being hardcoded. We used the application's built-in Ping feature to test Oracle and MongoDB connectivity separately before verifying end-to-end multi-database workflows.

11. TESTING

11.1 Test Plan

Test ID	Test Scenario	Test Input / Action	Expected Result
T01	Create vehicle with valid details	Enter valid registration, type, capacity, and active status.	Vehicle record is created in Oracle.
T02	Insert duplicate vehicle registration	Attempt inserting a vehicle with an existing registration number.	Oracle rejects insertion via unique constraint (DUP_VAL_ON_INDEX).
T03	Register passenger with duplicate email	Attempt inserting a passenger with an existing email address.	Oracle rejects insertion via unique constraint on email.
T04	Valid seat booking creation	Book an available seat for an existing passenger and scheduled trip.	Booking record is created and seat is allocated.
T05	Double-booking prevention	Attempt to book an already occupied seat number on the same trip.	CreateBooking procedure rejects the booking.
T06	Unavailable vehicle assignment	Schedule a trip using a vehicle with status 'Maintenance'.	ScheduleTrip procedure aborts and raises a custom error.
T07	Revenue calculation function	Execute CalculateRevenue for a specific date range.	Function returns the correct total sum of paid bookings.

Test ID	Test Scenario	Test Input / Action	Expected Result
T08	MongoDB full-text keyword search	Search reviewsFeedback with keywords like "clean" or "delay".	MongoDB text index returns matching review documents.
T09	Vehicle rating aggregation	Run aggregation pipeline grouping ratings by vehicle ID.	Returns ranked list of vehicles sorted by average rating.
T10	Dual database connection check	Trigger diagnostic ping from the backend.	Both Oracle and MongoDB connections report success.
T11	Update record via web interface	Edit a driver's contact number through the web form.	Updated phone number is saved in Oracle and appears on refresh.
T12	Form validation error handling	Submit booking form with empty passenger ID or negative fare.	Application displays an appropriate error message and prevents submission.

11.2 Testing Methods

- **Unit Testing:** Verified individual PL/SQL procedures and functions using dedicated test scripts in SQL Developer.
- **Database Constraint Testing:** Intentionally inserted invalid and duplicate records to verify that primary keys, unique constraints, and check constraints reject invalid data.
- **Functional Testing:** Tested end-to-end user workflows through the web browser, including creating bookings, updating drivers, and searching reports.
- **Regression Testing:** Re-ran core test scripts after making bug fixes to confirm existing features remained functional.

12. RESULTS

12.1 Database Results

- All 9 tables were successfully created in Oracle with primary keys, foreign keys, and validation check constraints.
- Test records from sample_data.sql were inserted without errors.
- Unique constraints successfully prevented duplicate registrations, duplicate emails, and double-booked seats.

12.2 PL/SQL Results

- CreateBooking, CancelBooking, ScheduleTrip, and RegisterPassenger procedures compiled and executed as expected.
- CalculateRevenue and GetAvailableSeats functions returned accurate results when tested against sample data.
- Exception handlers successfully trapped NO_DATA_FOUND and DUP_VAL_ON_INDEX, returning user-friendly error codes via RAISE_APPLICATION_ERROR.

12.3 MongoDB Results

The smartmove_transport database and all 4 collections were successfully initialized.
Indexes (unique, compound, and text) were verified using getIndexes().
Aggregation pipelines successfully computed average ratings by route, ranked vehicles and drivers, and counted keyword occurrences.

12.4 Application & Integration Results

Measurement Metric	Target Expectation	Verified Result	Status
Oracle tables created	9 relational tables	9 tables verified in schema browser	Pass
MongoDB collections created	4 document collections	4 collections active in smartmove_transport	Pass
PL/SQL procedures and functions	Stored procedures, functions, cursors, exceptions	All scripts tested in SQL Developer	Pass
SQL queries tested	DDL, DML, Joins, Aggregation	Tested in SQL Developer worksheet	Pass
MongoDB queries and aggregations	Keyword search, ratings ranking	Verified in mongosh and Compass	Pass
Web application CRUD pages	Vehicle, Driver, Route, Trip, Booking views	Tested in Google Chrome	Pass
Dual database integration	OCI8 and MongoDB PHP connections	Verified via diagnostic ping	Pass

13. DISCUSSION AND CONCLUSION

Developing SmartMove Transport Solutions gave our team practical experience in designing and implementing a hybrid database system. Rather than treating databases as simple data stores, we used Oracle's relational capabilities and constraints to protect operational integrity, and wrote PL/SQL stored procedures to encapsulate core business rules directly inside the database. At the same time, using MongoDB for passenger reviews, announcements, vehicle compliance files, and media demonstrated how document databases simplify handling unstructured content without complicating the relational schema.

13.1 Real Project Experiences & Challenges

- **Database Configuration:** Setting up the PHP OCI8 extension to communicate with Oracle XE required careful configuration of Oracle Instant Client environment variables. Testing database connections independently with a diagnostic ping script saved significant debugging time.
- **Stored Procedure Exception Handling:** During early testing of our PL/SQL scripts, we encountered minor syntax issues (such as a typo in a DBMS_OUTPUT call) which initially caused compilation warnings. Fixing these helped us understand how important clean exception handling is for debugging.
- **Date and Timestamp Formatting:** Aligning date formats between HTML input fields (YYYY-MM-DD), PHP, and Oracle DATE/TIMESTAMP columns required standardized formatting to avoid conversion errors.

13.2 Limitations & Lessons Learned

- **Concurrent Seat Booking:** In our current implementation, seat availability is checked right before inserting the booking. Under high concurrent traffic, two users could potentially pass the availability check at the exact same instant. While our composite unique constraint (TripID, SeatNumber) prevents the second insert, implementing pessimistic locking (SELECT ... FOR UPDATE) in the PL/SQL procedure would provide smoother handling.
- **Role-Based Access Control:** Currently, the web interface has basic administrative views. Adding strict role-based access control (distinguishing regular ticket clerks from fleet managers and administrators) would prevent unauthorized modifications.
- **Automated Document Expiry Alerts:** Setting up scheduled background jobs in MongoDB to alert managers when vehicle insurance or fitness certificates are within 30 days of expiry would be a valuable addition.

13.3 Future Work

Implement real-time GPS tracking integration for active vehicles using WebSockets.

Add automated SMS ticket confirmations to passengers upon successful booking.

Implement role-based authorization in the web application to separate cashier, manager, and administrator roles.

Introduce automated maintenance alerts triggered when a vehicle reaches its mileage service threshold.

14. REFERENCES

1. Android Developers. (2024). *Data storage overview & Room persistence library*. Available at: https://developer.android.com/training/data-storage/room (Accessed: 10 October 2026).
2. GitHub. (n.d.). *GitHub Documentation*. Available at: https://docs.github.com/ (Accessed: 10 October 2026).
3. MongoDB, Inc. (n.d.). *MongoDB Manual*. Available at: https://www.mongodb.com/docs/manual/ (Accessed: 10 October 2026).
4. Oracle. (n.d.). *Oracle Database Documentation*. Available at: https://docs.oracle.com/en/database/ (Accessed: 10 October 2026).
5. Oracle. (n.d.). *Oracle SQL Developer Data Modeler User Guide*. Available at: https://docs.oracle.com/en/database/oracle/sql-developer-data-modeler/ (Accessed: 10 October 2026).
6. PHP Documentation Group. (n.d.). *PHP Manual: OCI8 Functions and MongoDB Driver*. Available at: https://www.php.net/manual/en/ (Accessed: 10 October 2026).
7. SmartMove Transport Solutions. (2026). *SmartMove Transport Solutions Coursework Repository*. Available at: GitHub repository (Accessed: 10 October 2026).

APPENDICES

Appendix A: Oracle Table Creation Scripts

Complete DDL script from oracle/create_tables.sql defining all nine relational tables, primary keys, foreign keys, unique rules, and check constraints.

Appendix B: SQL Queries and Execution Outputs

Catalog of SQL report queries, join statements, and execution outputs demonstrating fleet status summaries and route booking statistics.

Appendix C: PL/SQL Procedures, Functions, Cursors, and Exception Tests

Full source code for CreateBooking, CancelBooking, ScheduleTrip, RegisterPassenger, stored functions (CalculateRevenue, CountPassengerTrips, GetAvailableSeats), cursors, and test scripts with DBMS_OUTPUT logs.

Appendix D: MongoDB Setup, Insertion, Query, and Aggregation Scripts

Complete scripts for the smartmove_transport database.

Appendix E: Application Screenshots

Screenshots of the running web application, including login, dashboard, vehicle/driver management, bookings, payments, and searchable reports.

Appendix F: Integration and Verification Evidence

Diagnostic connection ping results, .env configuration sample, and verification test outputs.

Appendix G: GitHub Repository Link

The complete source code, database scripts, and commit history for SmartMove Transport Solutions are available at:
https://github.com/chanura-madushan/SmartMove-Transport-Solutions